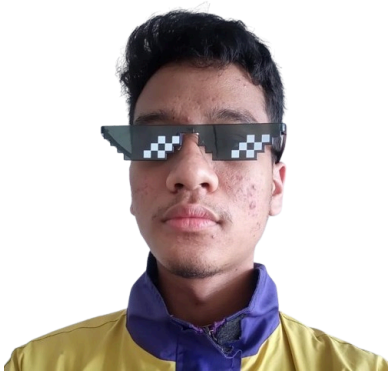
Laporan Pengerjaan Tugas 3

Aplikasi Chat Web dengan Kriptografi Kunci-Simetri dan Kunci-Publik

Diajukan untuk memenuhi tugas mata kuliah II4021 Kriptografi pada Semester VI tahun Akademik 2025/2026



Disusun oleh:

Thaffariq Azka Rahmat	18223048
Michael Ballard Isaiah Silaen	18223080
Muhammad Rafi Ramadhan	18223094

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

DAFTAR ISI 1
DAFTAR GAMBAR 2
BAB I TEORI DASAR 3
1.1. JSON Web Token (JWT) 3
1.2 Elliptic Curve Diffie-Hellman 4
1.3 AES 6
BAB II PERANCANGAN DAN IMPLEMENTASI 10
2.1. Perancangan Program 10
2.1.1. Arsitektur 10
2.1.2. Layanan Kriptografi 12
2.1.3. UI UX 12
2.2. Implementasi Program 14
2.2.1. Struktur Folder 14
2.2.2. Front End 14
2.2.3. Back End 15
2.2.4. Implementasi Kriptografi & Alur Kerja 16
2.2.5. UI UX 17
BAB III PENGUJIAN PROGRAM DAN ANALISIS HASIL 21
3.1. Uji Autentikasi dan Manajemen Pengguna 21
3.1.1. Registrasi dengan Data Valid 21
3.1.2. Registrasi dengan Email Terdaftar 22
3.1.3. Login dengan Password yang Benar 23
3.1.4. Login dengan Password yang Salah 24
3.2. Uji Implementasi dan JSON Web Token (JWT) 24
3.3. Uji Pembentukan Kunci Komunikasi (ECDH dan KDF) 25
3.4. Uji Enkripsi dan Dekripsi Pesan 26
3.5. Uji Integritas dan Autentikasi Pesan (MAC) (Bonus) 28
BAB IV KESIMPULAN 29
DAFTAR PUSTAKA 30
LAMPIRAN 31
Lampiran 1. Pernyataan Integritas 31
Lampiran 2. Pranala Repositori Github 31
Lampiran 3. Pranala Video Demo 31
Lampiran 4. Pembagian Tugas 31

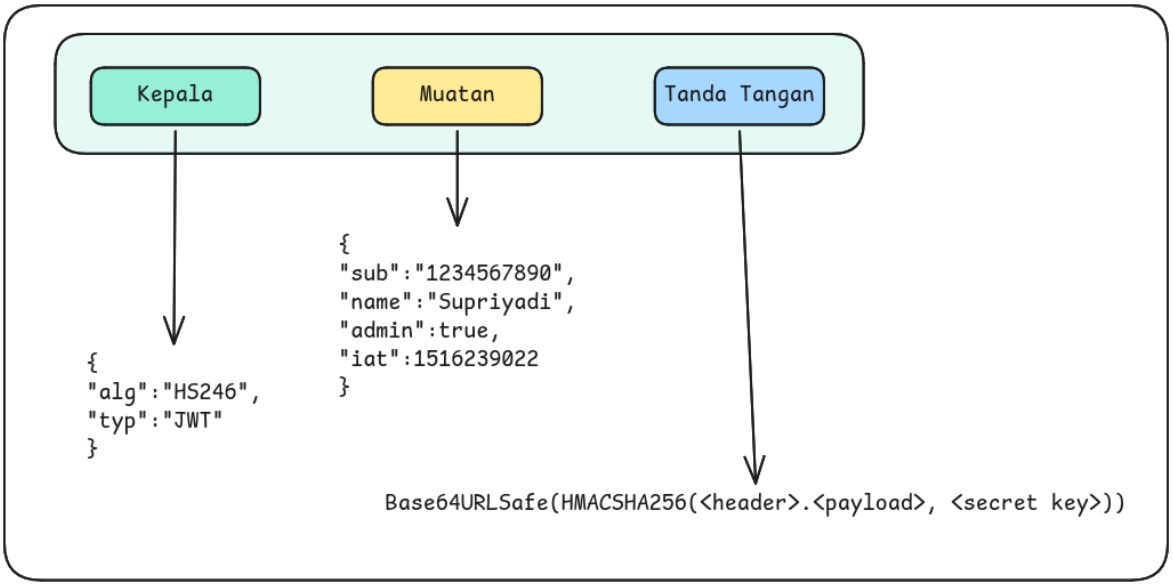
DAFTAR GAMBAR

Gambar 1.1 Struktur JWT..... 3
Gambar 1.2 Autentikasi JWT..... 4
Gambar 1.3 Operasi Penjumlahan Titik..... 5
Gambar 1.4 Ekspansi Kunci AES-256..... 6
Gambar 1.5 Operasi pada AES..... 7
Gambar 2.1 Diagram arsitektur aplikasi..... 10
Gambar 2.2 Pembagian Fungsi dengan Pattern MVC..... 11
Gambar 2.3 Halaman Login/Sign Up..... 12
Gambar 2.4 Halaman Utama..... 13
Gambar 2.5 Halaman Kontak..... 13
Gambar 2.6 Halaman Percakapan..... 14
Gambar 2.7 Landing Page..... 18
Gambar 2.8 Register Page..... 18
Gambar 2.9 Discover Users Page..... 19
Gambar 2.11 Chatroom..... 20
Gambar 2.12 After Sending Message..... 20

BAB I
TEORI DASAR

1.1. JSON Web Token (JWT)

JSON Web Token adalah cara aman untuk mengirim informasi antara sebuah klien dan server. Secara gamblang, JWT hanyalah sebuah paket JSON yang diamankan dengan tanda tangan kriptografis. JWT memiliki struktur sebagai berikut:



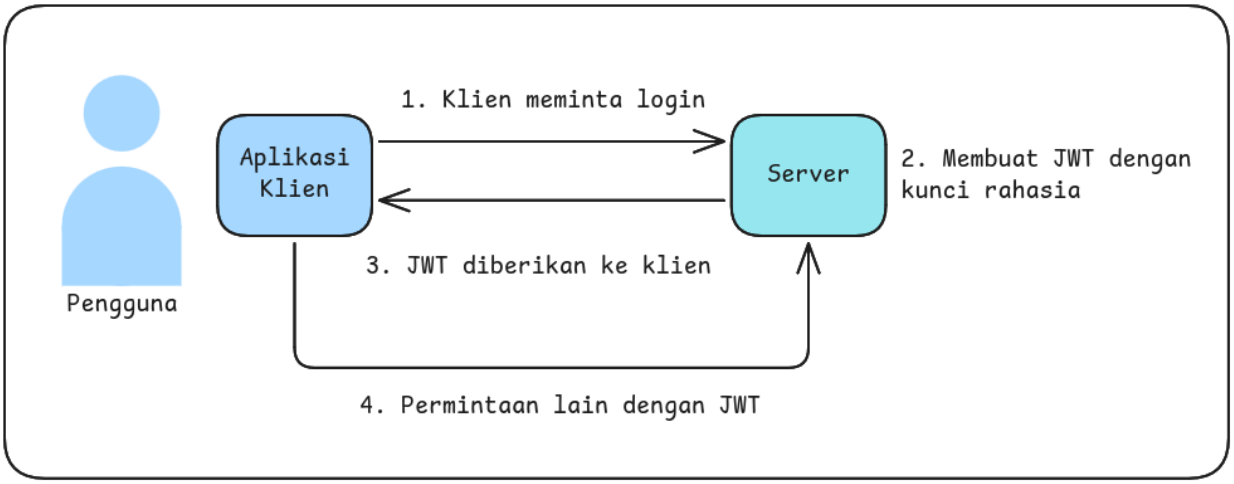
Gambar 1.1 Struktur JWT

Ketiga bagian ini dipisahkan dengan titik

Header.Payload.Signature

1. Header (Kepala) berisi metadata tentang token seperti algoritma yang dipakai untuk penandatanganan
2. Payload (Muatan) berisi klaim
    a. iss (Issuer) mengidentifikasi siapa klien pembuat token
    b. sub (Subject) mengidentifikasi pihak yang disebut di dalam token
    c. aud (Audience) mengidentifikasi pihak penerima token
    d. exp (Expiration) menunjukkan waktu kadaluarsa token
    e. nbf (Not Before) adalah kebalikan klaim exp, menunjukkan kapan token mulai berlaku
    f. iat (Issued at Time) menunjukkan waktu pembuatan token
    g. jti (JWT ID) adalah nomor identifikasi ID (agar bisa token-token bisa dibedakan)
3. Signature (Tanda tangan) adalah tanda tangan kriptografis

Cara kerja autentikasi menggunakan JWT kira-kira sebagai berikut:



Gambar 1.2 Autentikasi JWT

1.2 Elliptic Curve Diffie-Hellman

Kriptografi kunci simetris mengharuskan kedua belah pihak untuk menggunakan kunci yang sama karena kunci tersebut dipakai untuk mengenkripsi sekaligus mendekripsi pesan. Masalahnya adalah bagaimana cara mengirimkan kunci itu secara aman. Saluran yang ada untuk bertukar pesan seringkali tidak aman, itu sebabnya kriptografi dipakai. Kalau kunci berhasil ditangkap oleh pihak yang tidak diinginkan, mereka bisa mendekripsi pesan sehingga enkripsi percuma. Maka digunakanlah pertukaran kunci Diffie-Hellman.

Secara konseptual, pertukaran Diffie-Hellman dilakukan dengan memakai sebuah operasi yang memiliki sifat komutatif. Artinya urutan operasi tidak akan memengaruhi hasil akhir operasi selama semua operan yang terlibat sama. Contoh paling mudah adalah operasi penjumlahan. Apabila pertukaran dilakukan dengan penjumlahan, setiap pihak cukup menyiapkan angka untuk dirinya sendiri, mempublikasikan angka tersebut, kemudian mengambil angka dari pihak lain yang ingin berkomunikasi dan menjumlahkannya.

1. Alice memilih angka A
2. Bob memilih angka B
3. Alice mendapat angka B dari saluran publik dan melakukan A + B
4. Bob mendapat angka A dari saluran publik dan melakukan B + A

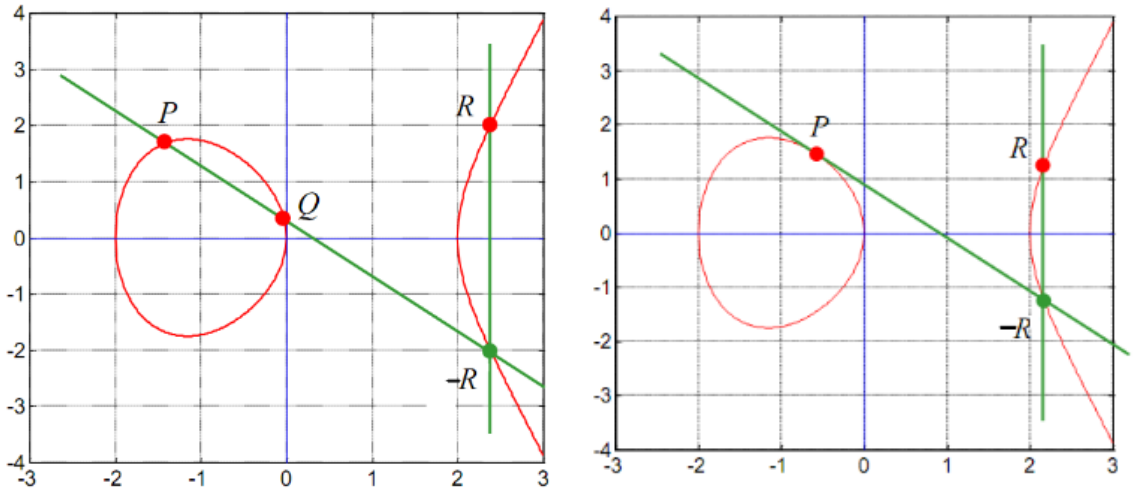
Oleh karena sifat komutatif tersebut, baik Alice dan Bob akan mendapat angka yang sama persis. Masalahnya pada metode ini, operasi penjumlahan terlalu mudah untuk dibalik. Siapapun bisa dengan mudah melihat A dan B di saluran publik dan mendapat kunci yang sama. Untuk menangani masalah ini, operasi yang dipilih untuk melakukan pertukaran kunci haruslah memiliki masalah logaritma diskrit. Salah satu operasi tersebut melibatkan kurva eliptik.

Kurva Eliptik merupakan sebuah kurva dengan persamaan berikut:

$$y^2 = x^3 + ax + b$$

Kurva ini memiliki sebuah properti spesial berupa adanya masalah logaritma diskrit. Artinya ada suatu operasi yang mudah dilakukan secara "maju" tetapi sukar dilakukan secara "mundur", operasi itu adalah penjumlahan titik. Pada kurva eliptik, sebuah titik bisa "dijumlahkan" dengan titik lain untuk mendapatkan koordinat suatu titik baru dengan cara geometris berikut:

1. Tarik garis yang melalui kedua titik (kalau titik dijumlahkan pada diri sendiri berarti carilah garis singgung di titik itu)
2. Lihat dimana lagi garis itu memotong kurva, titik perpotongan baru ini disebut -R
3. Cerminkan -R terhadap sumbu X, akan didapat titik baru, ini adalah R, titik hasil penjumlahan



Gambar 1.3 Operasi Penjumlahan Titik

Penjumlahan titik ini bisa dilakukan berkali-kali sampai akhirnya titik R akan kembali ke titik semula. Artinya pada sebuah kurva eliptik, ada sekumpulan titik yang jumlahnya berhingga yang bisa "dikunjungi" dengan cara menambahkan suatu titik di kumpulan itu terhadap dirinya sendiri. Selain sifat tersebut, mencari penjumlahan titik sebanyak N kali sangat mudah, akan tetapi menembak berapa kali penjumlahan yang diperlukan agar mencapai titik tersebut sangat sulit. "Mudah dilakukan secara maju, sulit dilakukan secara mundur", semua sifat ini membuat kurva eliptik sangat bisa dipakai untuk kriptografi.

Salah satu aplikasinya untuk pertukaran kunci Diffie-Hellman, disebut ECDH (*Elliptic Curve Diffie-Hellman*). Alurnya adalah sebagai berikut:

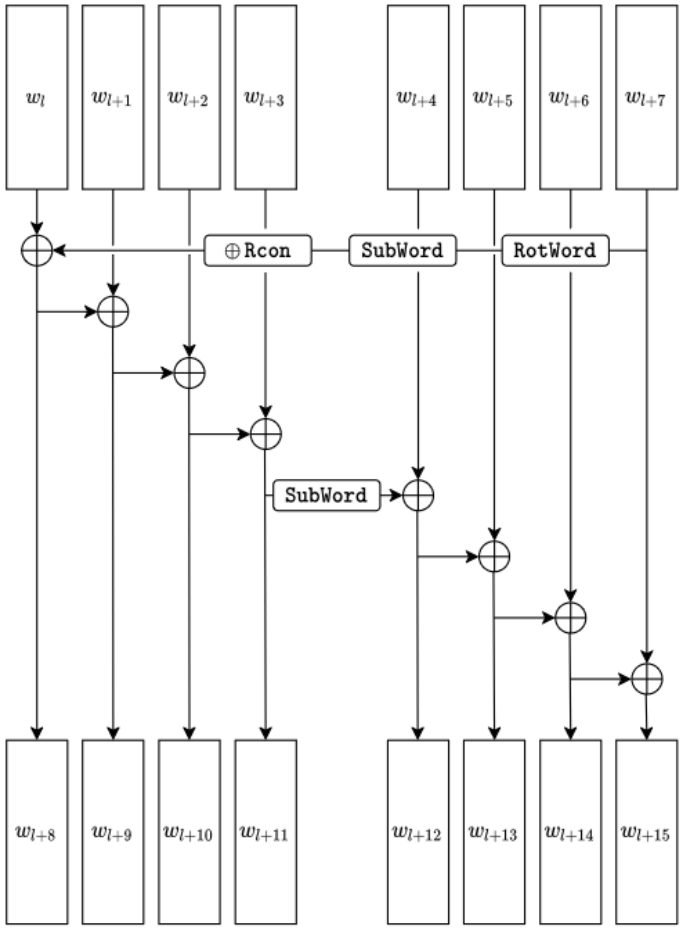
1. Alice dan Bob menyepakati sebuah kurva eliptik dan sebuah titik awal P
2. Alice memilih angka A, melakukan A * P lalu mengirim AP ke Bob
3. Bob memilih angka B, melakukan B * P lalu mengirim BP ke Alice
4. Alice tinggal menghitung A * (BP)

5. Bob tinggal menghitung B * (AP)
6. Keduanya mendapat sebuah koordinat titik yang sama, ABP = BAP karena AB = BP

1.3 AES

AES (*Advanced Encryption Standard*) adalah standar enkripsi yang melakukan kriptografi simetris. Algoritma yang dipakai bernama algoritma Rijndael. Algoritma ini dikembangkan sebagai respons terhadap berhasilnya serangan terhadap DES (*Data Encryption Standard*). Secara umum AES memiliki dua tahap: ekspansi kunci dan enkripsi/dekripsi. Pesan biasanya jauh lebih panjang daripada kunci, untuk menangani kriptanalisis dengan metode seperti metode Kasiski, diperlukan kunci yang panjangnya sama dengan panjang pesan dan kelihatannya acak (pseudorandom). Ini alasannya ada ekspansi kunci.

Ekspansi kunci pada AES bergantung pada panjang kunci awal. Pada aplikasi ini, tipe yang dipakai adalah kunci sepanjang 256 bit. Berikut adalah proses ekspansi kunci pada AES-256:



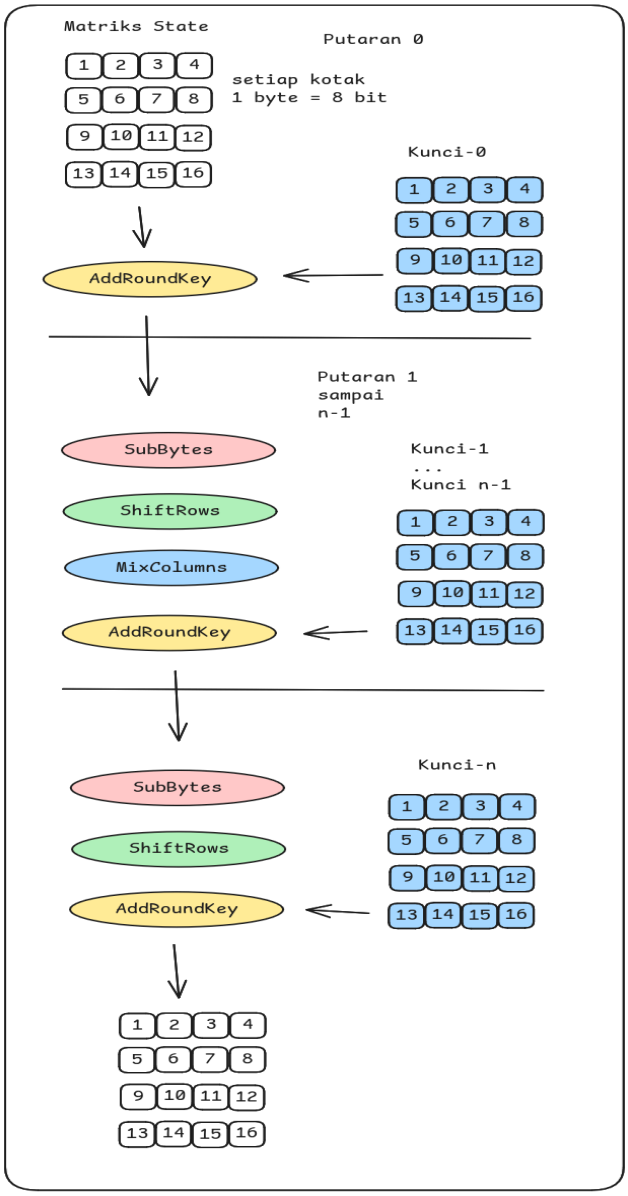
Gambar 1.4 Ekspansi Kunci AES-256

1. Kunci awal sepanjang 32 byte akan dibagi menjadi 8 word (1 word = 4 byte)

2. Setiap word dioperasikan sesuai dengan gambar di atas
3. Rcon artinya di XOR dengan Round Constant
4. SubWord artinya word tersebut disubstitusi dengan S-Box
5. RotWord artinya byte-byte pada sebuah word diputar sekali ke kiri secara melingkar

Operasi-operasi ini berlangsung sampai diperoleh 15 kunci putaran.

Setelah kunci putaran didapat, operasi enkripsi/dekripsi bisa dilakukan.



Gambar 1.5 Operasi pada AES

Secara umum AES melakukan 4 operasi yang sama berkali-kali. Operasi tersebut adalah:

1. SubBytes:
   Mengganti sebuah byte dengan byte lain dari tabel S-Box
2. ShiftRows:
   Menggeser baris-baris state secara melingkar ke kiri sejumlah index baris
3. MixColumns:
   Mengalikan setiap kolom dengan sebuah matriks khusus di dalam Galois Field
4. AddRoundKey:
   Melakukan XOR setiap byte pada state dengan kunci putaran hasil ekspansi sebelumnya

Pada enkripsi AES 256, terdapat 15 putaran/ronde (dari 0-14).

1. Ronde 0:
   a. AddRoundKey
2. Ronde 1 sampai 13:
   a. SubBytes
   b. ShiftRows
   c. MixColumns
   d. AddRoundKey
3. Ronde terakhir (14):
   a. SubBytes
   b. ShiftRows
   c. AddRoundKey

Dekripsi pada AES 256 menggunakan invers (kebalikan) dari fungsi-fungsi enkripsi:

1. InvSubBytes:
   Mengganti sebuah byte dengan byte lain dari tabel INVS-Box
2. ShiftRows:
   Menggeser baris-baris state secara melingkar ke kanan sejumlah index baris
3. InvMixColumns:
   Mengalikan setiap kolom dengan sebuah matriks khusus (yang berbeda dari matriks khusus pada MixColumns) di dalam Galois Field
4. AddRoundKey:
   Melakukan XOR setiap byte pada state dengan kunci putaran hasil ekspansi sebelumnya (invers dari XOR adalah XOR sehingga operasi tidak perlu diubah)

Pada dekripsi AES 256, terdapat 15 putaran/ronde (dari 0-14), urutan operasinya pun berbeda dari enkripsi.

1. Ronde 0:
    a. AddRoundKey
2. Ronde 1 sampai 13:
    a. InvShiftRows
    b. InvSubBytes
    c. AddRoundKey
3. Ronde terakhir (14):
    a. InvShiftRows
    b. InvSubBytes
    c. AddRoundKey

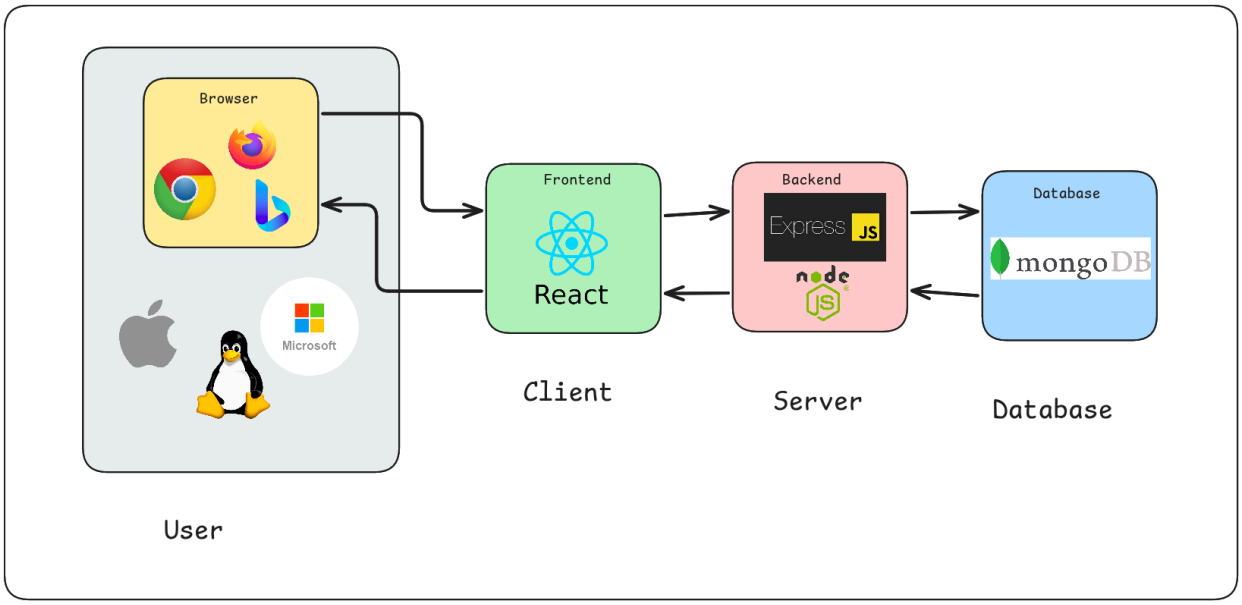
BAB II
PERANCANGAN DAN IMPLEMENTASI

2.1. Perancangan Program

2.1.1. Arsitektur

Arsitektur lapis yang digunakan untuk aplikasi ini adalah arsitektur lapis-3 atau 3-tier. Aplikasi dipisah menjadi 3 bagian:

1. Client/Frontend: Berisi fitur chat, enkripsi, dan semua fungsionalitas terkait
2. Server/Backend: Berfungsi untuk membuat JWT
3. Database: Berfungsi menyimpan database user (informasi login dan sebagainya)

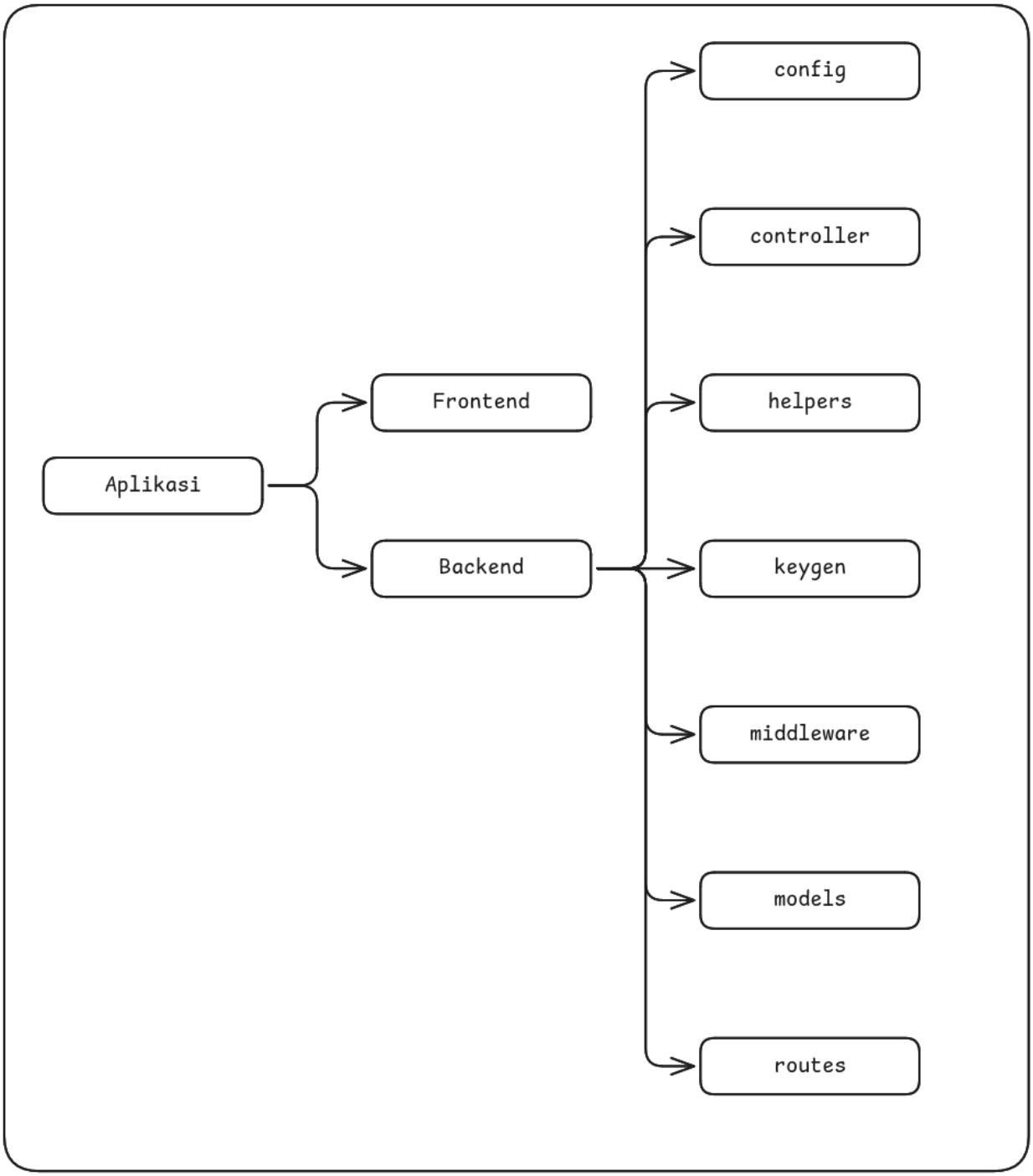


Gambar 2.1 Diagram arsitektur aplikasi

Alasan pemisahan lapisan seperti yang telah dijabarkan adalah kebutuhan yang dicantumkan dalam spesifikasi. Server tidak boleh memegang kunci komunikasi maupun mengetahui polosan pesan. Dengan kata lain, server hanyalah perantara yang "buta" untuk autentikasi dan penerusan data. Untuk mengakomodasi kebutuhan tersebut, semua fungsi yang berkaitan dengan enkripsi dan dekripsi pesan dipasang di bagian klien. Sementara itu, JWT ada di server (untuk memenuhi kebutuhan autentikasi tadi).

Stack teknologi yang dipilih untuk proyek ini adalah stack MERN (MongoDB, Express, React, Node). Stack ini dipilih karena relatif sederhana dan mudah dipakai. Framework yang dipakai adalah Vite karena menyediakan pemisahan yang sesuai dengan kebutuhan arsitektur ini. Frontend dan backend sudah dipisah dari awal dan koneksi ke database sangat mudah dilakukan (cukup menambahkan konfigurasi koneksi di backend saja).

Pada codebasenya sendiri, pemisahan lebih jauh akan diimplementasikan seperti ini:



Gambar 2.2 Pembagian Fungsi dengan Pattern MVC

Pembagian fungsi ini (ditambah beberapa folder tambahan)

1. Frontend akan berisi semua fitur yang langsung berinteraksi dengan pengguna (termasuk enkripsi/dekripsi untuk chat)

2. Folder config akan berisi konfigurasi koneksi ke database
3. Folder controller akan berisi pengontrol yang akan memanipulasi model
4. Folder helper akan berisi fungsi-fungsi pembantu (termasuk fungsi kriptografis seperti pada proses tanda tangan)
5. Folder keygen akan berisi fungsi pembangkitan kunci
6. Folder middleware akan berisi fungsi perantara (utamanya yang dipakai untuk autentikasi)
7. Folder model akan berisi semua model (user, chat, dan message)
8. Folder routes akan berfungsi menunjukkan aset-aset

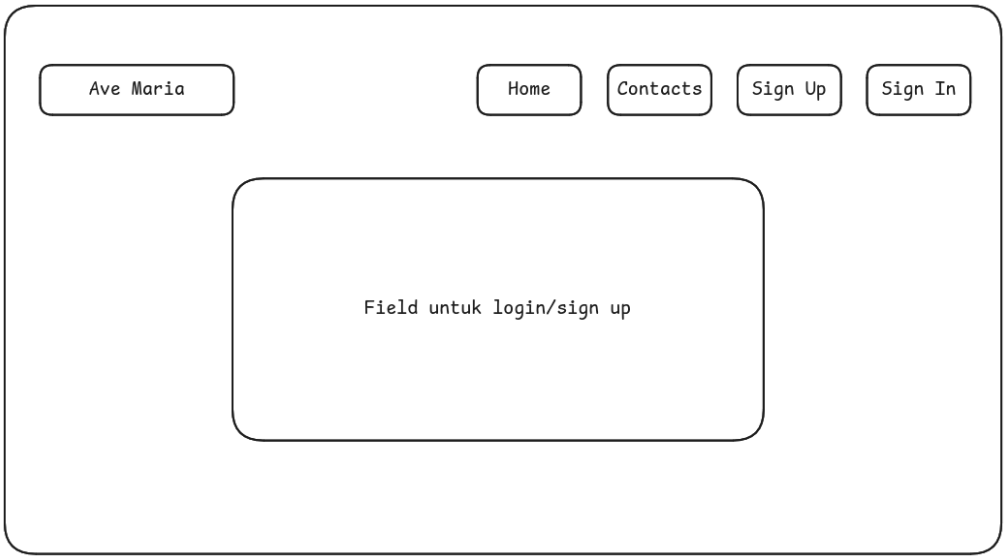
2.1.2. Layanan Kriptografi

Layanan kriptografi yang disediakan oleh aplikasi ini adalah:

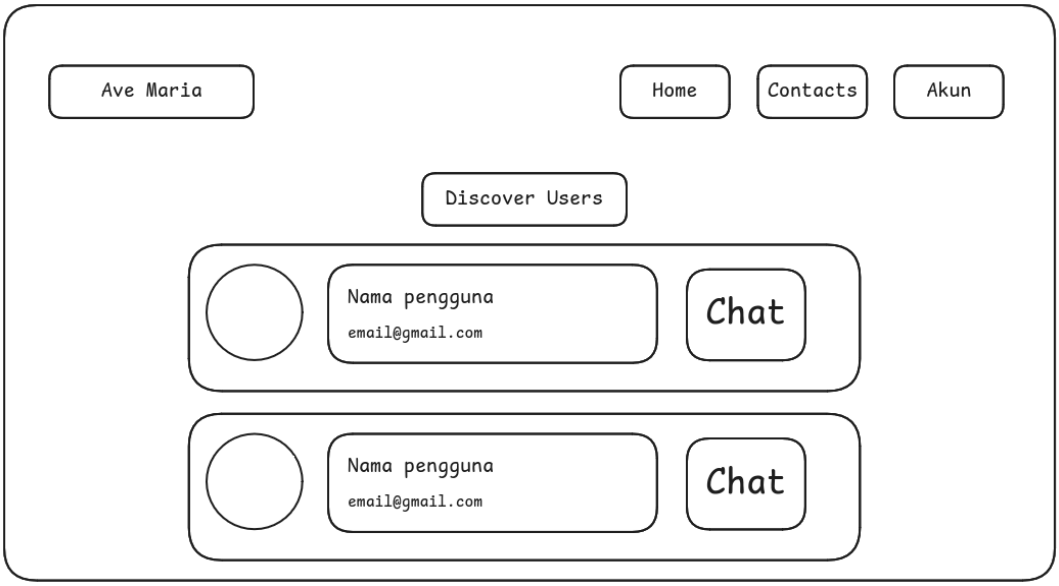
1. Autentikasi dan penandatanganan dengan ECDSA (kurva yang dipakai adalah P-256)
2. Pertukaran kunci menggunakan Elliptic Curve Diffie-Hellman (kurva yang dipakai adalah P-256)
3. Enkripsi simetris pesan menggunakan AES-256 mode GCM

Proses pengiriman pesan juga dilakukan sesuai urutan tersebut. Autentikasi dengan JWT akan difasilitasi oleh ECDSA. Setelah autentikasi berhasil, pengguna menyepakati kunci AES bersama menggunakan ECDH. Terakhir, kunci tersebut dipakai untuk mengenkripsi pesan percakapan.

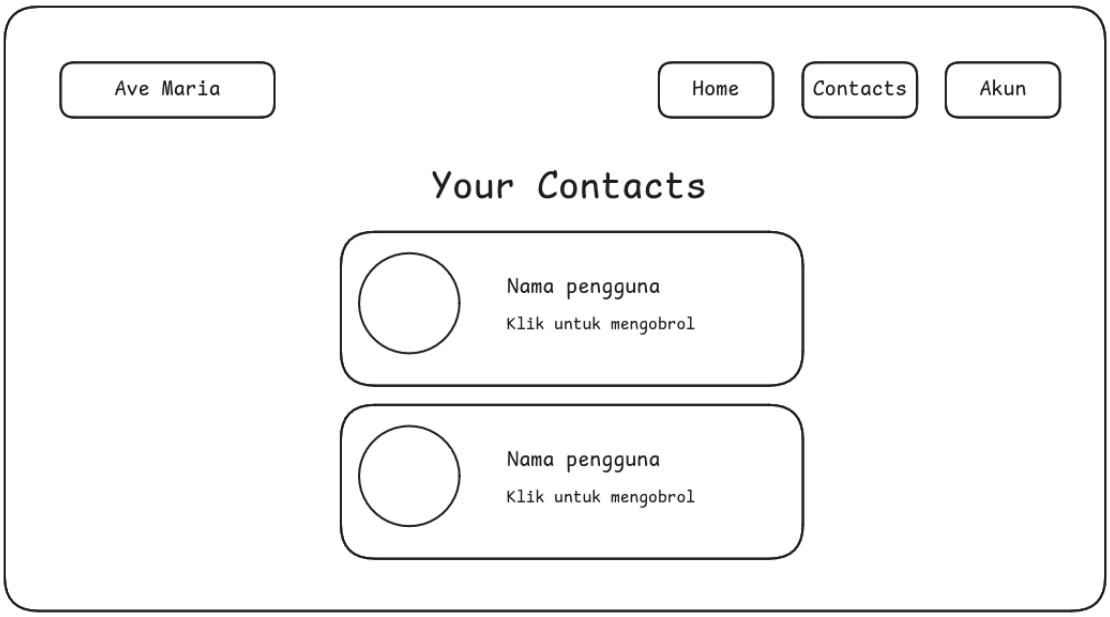
2.1.3. UI UX



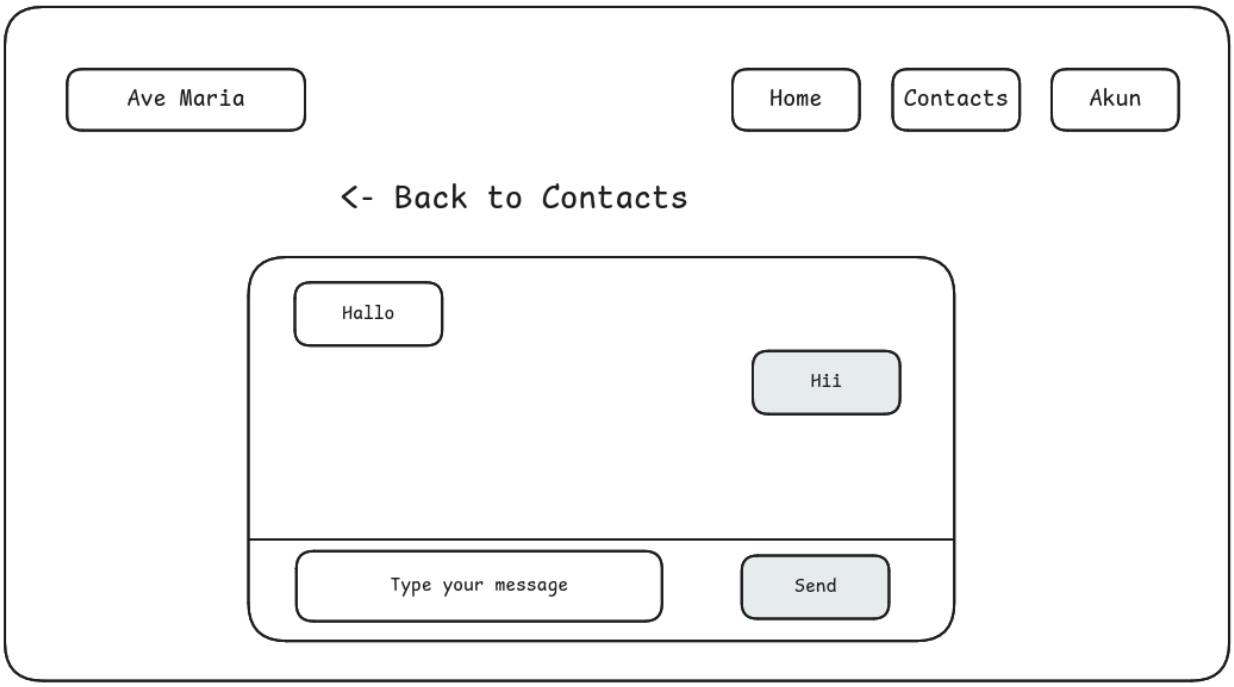
Gambar 2.3 Halaman Login/Sign Up



Gambar 2.4 Halaman Utama



Gambar 2.5 Halaman Kontak



Gambar 2.6 Halaman Percakapan

2.2. Implementasi Program

2.2.1. Struktur Folder

```
Tugas3Kriptografi-main/
├── chatbackend/       # Bagian Server-side (Node.js & Express)
├── chatfrontend/      # Bagian Client-side (React & Vite)
└── docker-compose.yml
```

2.2.2. Front End

```
chatfrontend/
├── public/            # Aset statis (logo, ikon)
├── src/               # Kode sumber aplikasi utama
│   ├── components/    # Komponen UI (Navbar, Footer, Universal)
│   ├── lib/           # Fungsi integrasi API dan Global State
│   │   ├── actions/   # Logika Axios dan Error Handling
│   │   ├── contexts/  # State management (User Context)
│   │   ├── CustomHooks/# Fungsi React kustom (useFetch)
│   │   └── types/     # Definisi tipe data TypeScript
│   ├── pages/         # Halaman utama (Chat, Login, Home, dsb.)
│   ├── routes/        # Sistem navigasi aplikasi (React Router)
│   ├── App.tsx        # Komponen utama pembungkus aplikasi
│   ├── main.tsx       # Titik masuk utama (Entry point)
│   └── index.css      # Styling global
```

```
├── tailwind.config.js  # Konfigurasi desain Tailwind CSS
├── tsconfig.json       # Konfigurasi TypeScript
├── vite.config.ts      # Konfigurasi build tool Vite
├── package.json        # Daftar dependensi frontend
└── Dockerfile
```

Bagian Frontend (chatfrontend/) berfokus pada Visual dan Aset UI yang berfungsi sebagai antarmuka dari aplikasi. Isinya adalah aset-aset yang berurusan langsung dengan interaksi pengguna:

- Aset UI Reusable: Di dalam folder components, terdapat komponen dasar seperti tombol, input teks, dan modal. Ini adalah blok bangunan untuk membentuk tampilan.
- Halaman (Pages): Kumpulan dari komponen-komponen UI yang disusun menjadi satu tampilan utuh (seperti halaman chat atau halaman login).
- Styling: Menggunakan Tailwind CSS untuk mengatur estetika tampilan agar terlihat modern dan responsif.
- Logika Klien: Hanya menangani bagaimana data ditampilkan dan bagaimana mengirim input pengguna ke server.

2.2.3. Back End

```
chatbackend/
├── config/            # Pengaturan koneksi database
├── controllers/       # Logika utama (Auth, Chat, Message, User)
├── helpers/           # Fungsi bantuan (Hashing, Token, Casing)
├── keygen/            # Generator kunci kriptografi
├── middlewares/       # Filter keamanan (Auth & Error Handling)
├── models/            # Definisi skema data (Mongoose)
├── routes/            # Definisi jalur API (Endpoint)
├── app.js             # File utama eksekusi server
├── initialSeeding.js  # Script pengisian data awal
├── package.json       # Daftar dependensi backend
├── README.md          # Dokumentasi backend
└── Dockerfile
```

Bagian Backend (chatbackend/) berfungsi untuk mengolah data dengan cara menjalankan perintah di balik layar. Berikut adalah fungsi dari masing-masing foldernya:

- config/: Tempat menyimpan pengaturan koneksi ke database MongoDB, agar server tahu tempat menyimpan data.

- controllers/: Tempat diletakkannya logika bisnis. Misalnya, ketika user menekan tombol "Login", controller inilah yang menentukan apakah user boleh masuk atau tidak setelah memeriksa database.
- helpers/: Berisi alat-alat pendukung. Di sini terdapat fungsi untuk menyamarkan password (bcrypt) dan pembuatan token akses (JWT) agar sesi pengguna tetap aman.
- keygen/: Folder ini berisi skrip khusus untuk menghasilkan kunci-kunci kriptografi yang diperlukan untuk pengamanan pesan.
- middlewares/: Berperan mengecek permintaan sebelum masuk ke controller. Misalnya: "Apakah user ini sudah login?", "Apakah datanya valid?". Jika tidak, permintaan akan ditolak di sini.
- models/: Berisi schema data. Folder ini mendefinisikan bentuk data User itu seperti apa (nama, email, password), dan bentuk data Chat atau Message itu seperti apa di database.
- routes/: Berfungsi sebagai pintu masuk atau alamat. Folder ini memetakan URL (misalnya /api/login atau /api/messages) ke controller yang sesuai agar permintaan dari frontend sampai ke tujuan yang tepat.

Pemisahan tersebut membuat program menjadi lebih modular karena bagian tampilan tidak akan terganggu jika ada perubahan pada logika database, dan begitu pula sebaliknya.

2.2.4. Implementasi Kriptografi & Alur Kerja

```
chatbackend/
├── controllers/
│   └── MessageController.js    # Pengelola alur kirim/terima pesan terenkripsi
├── helpers/
│   ├── bcrypt.js               # Pengacak (hashing) password pengguna
│   └── jwt.js                  # Pembuat & verifikator token akses (EC-based)
├── keygen/
│   └── generateKeys.js         # Generator pasangan kunci (Public/Private) untuk JWT
├── middlewares/
│   └── Authentication.js       # Verifikator identitas pengguna sebelum akses data
└── models/
    ├── User.js                 # Penyimpan kunci publik & kunci privat terenkripsi
    ├── Chat.js                 # Pengelola daftar peserta percakapan sah
    └── Message.js              # Penyimpan ciphertext, IV, dan MAC (bukan teks asli)
```

A. Objek Data (Models)

- models/User.js: File ini menyimpan identitas kriptografi pengguna, termasuk publicKey dan encryptedPrivateKey.
- models/Chat.js: Bertugas memastikan integritas hubungan antar pengguna, sehingga hanya participants yang sah yang terhubung dengan ID percakapan tertentu.
- models/Message.js: Menyimpan data teknis kriptografi hasil enkripsi dari sisi klien, yaitu ciphertext, iv (Initialization Vector), dan mac.

B. Autentikasi & Keamanan Sesi

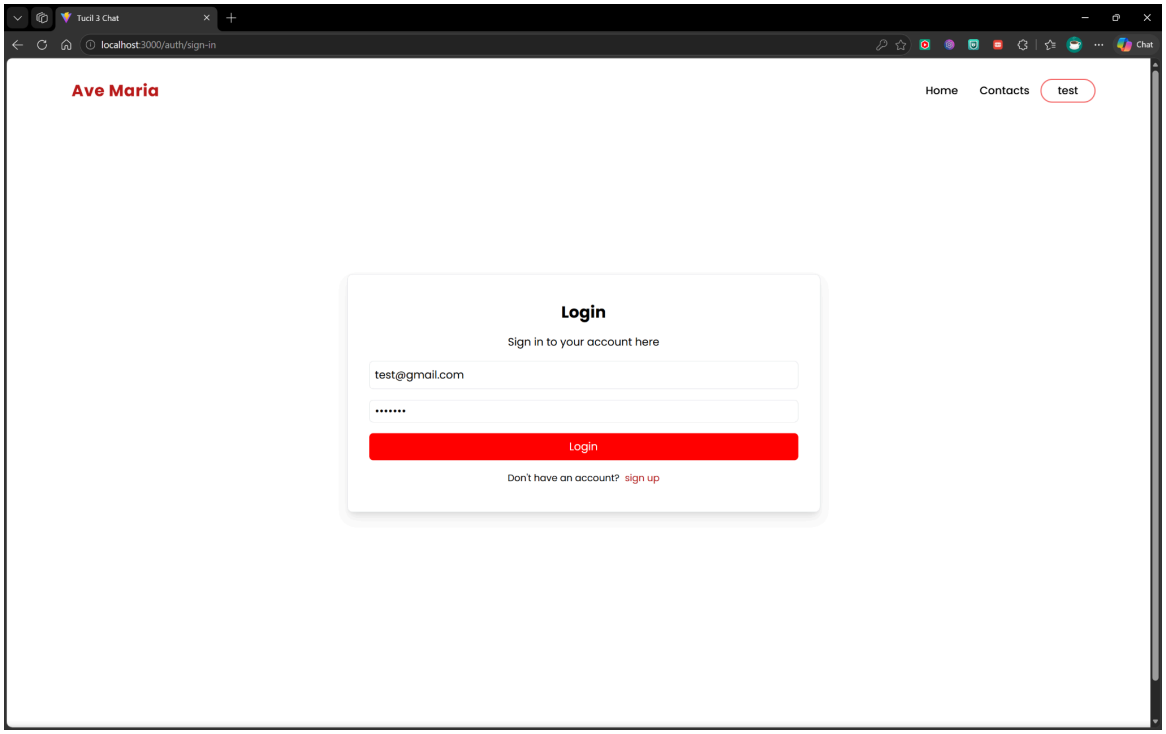
- keygen/generateKeys.js: Skrip ini menggunakan algoritma Elliptic Curve (prime256v1) untuk menghasilkan kunci yang digunakan pada level sistem (JWT), bukan sekadar enkripsi teks biasa.
- helpers/jwt.js & helpers/bcrypt.js: bcrypt memastikan password tidak tersimpan dalam bentuk teks polos, sementara jwt.js menangani pembuatan token digital yang aman menggunakan kunci dari proses keygen.
- middlewares/Authentication.js: Berfungsi memeriksa apakah token JWT yang dibawa pengguna valid sebelum mereka diizinkan memanggil fungsi di MessageController.

C. Logika Alur Kerja (Controller)

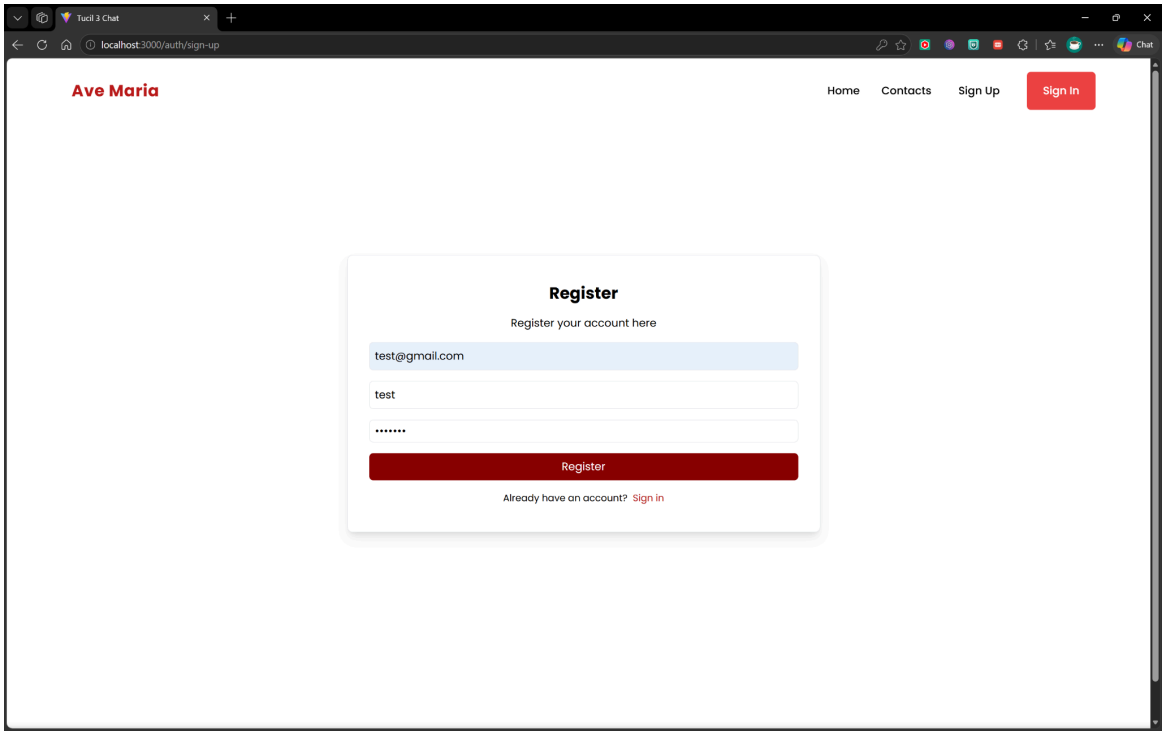
- controllers/MessageController.js:
- Saat Kirim Pesan (createMessage): Memastikan data wajib seperti chatId, ciphertext, dan iv tersedia, lalu memverifikasi bahwa pengirim adalah bagian dari percakapan tersebut sebelum disimpan ke database.
- Saat Terima Pesan (getMessages): Mengambil data terenkripsi dari database berdasarkan ID percakapan, setelah memastikan pengguna yang meminta memiliki hak akses.

Struktur ini menjamin bahwa keamanan tidak hanya terletak pada satu file, melainkan tersebar mulai dari pendefinisian data di Models, penyaringan akses di Middlewares, hingga eksekusi logika di Controllers.

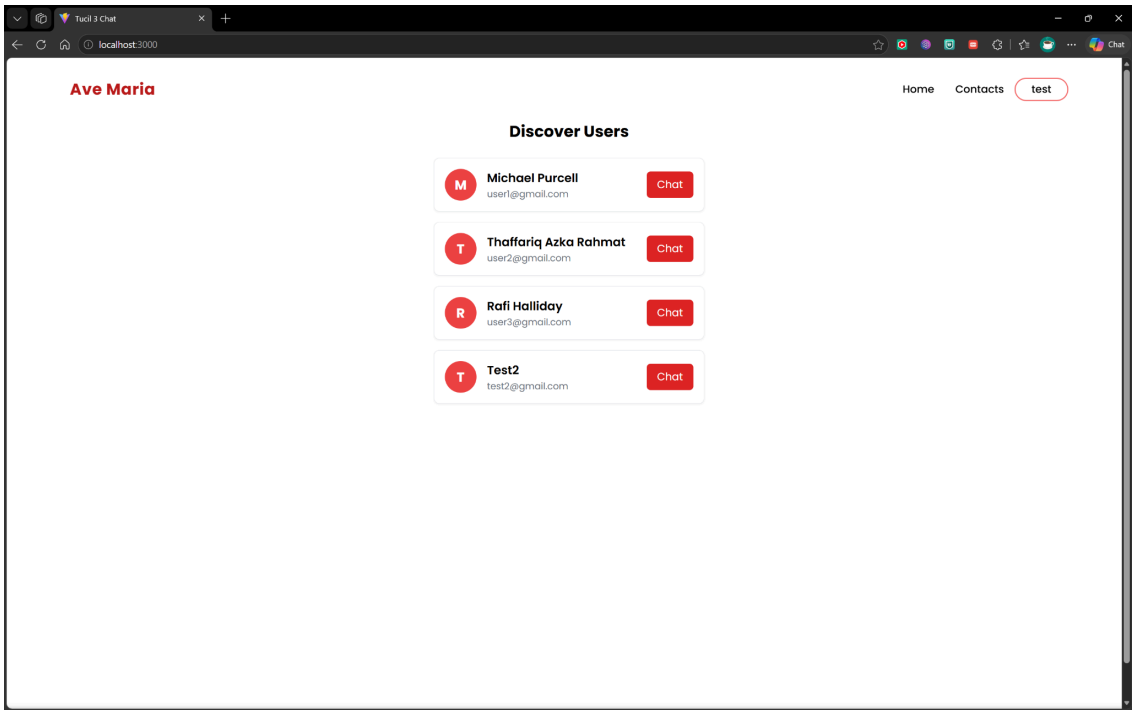
2.2.5. UI UX



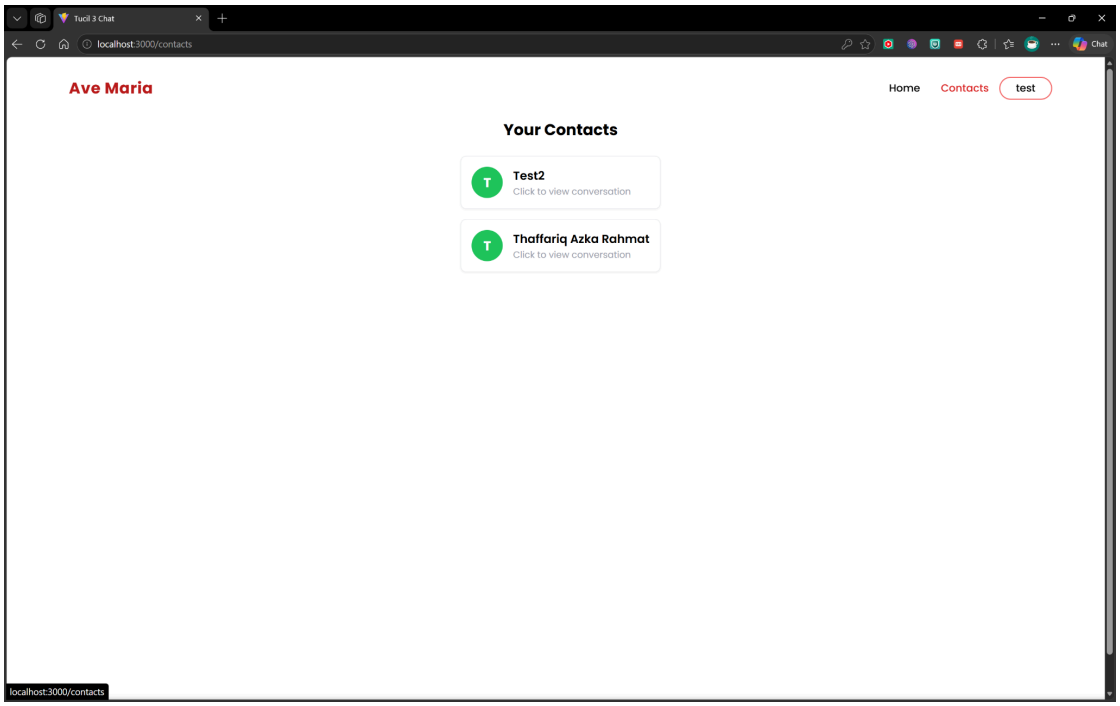
Gambar 2.7 Landing Page



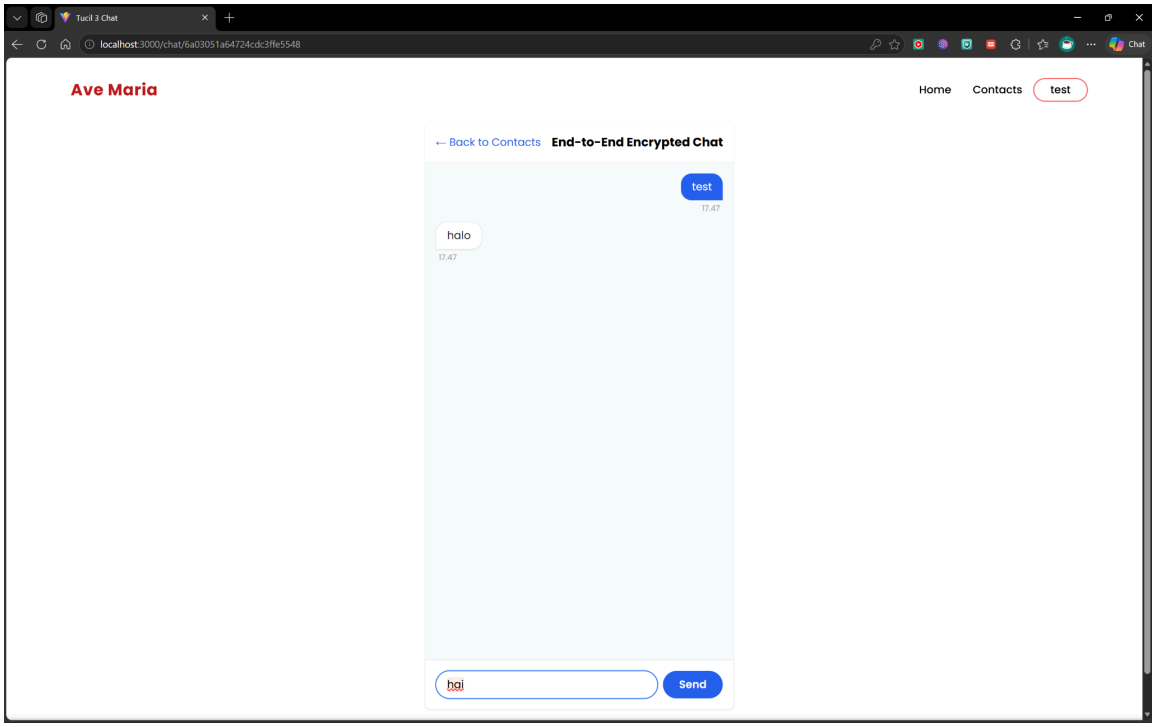
Gambar 2.8 Register Page



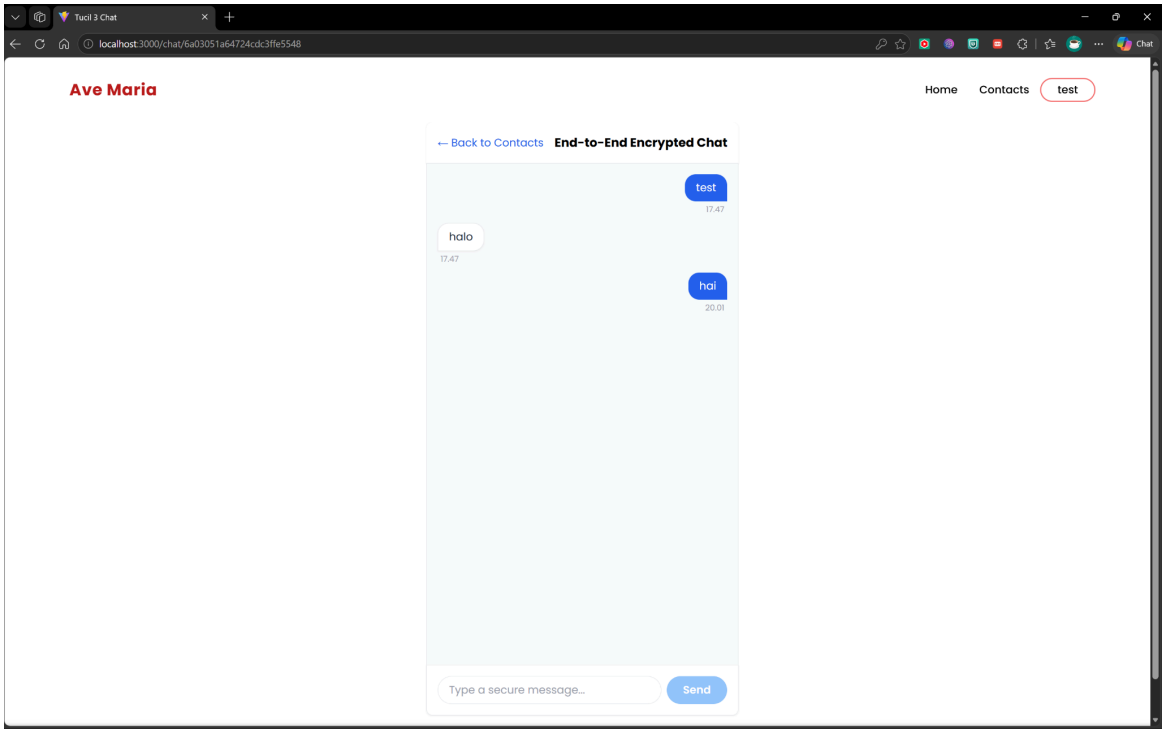
Gambar 2.9 Discover Users Page



Gambar 2.10 Contacts Page



Gambar 2.11 Chatroom



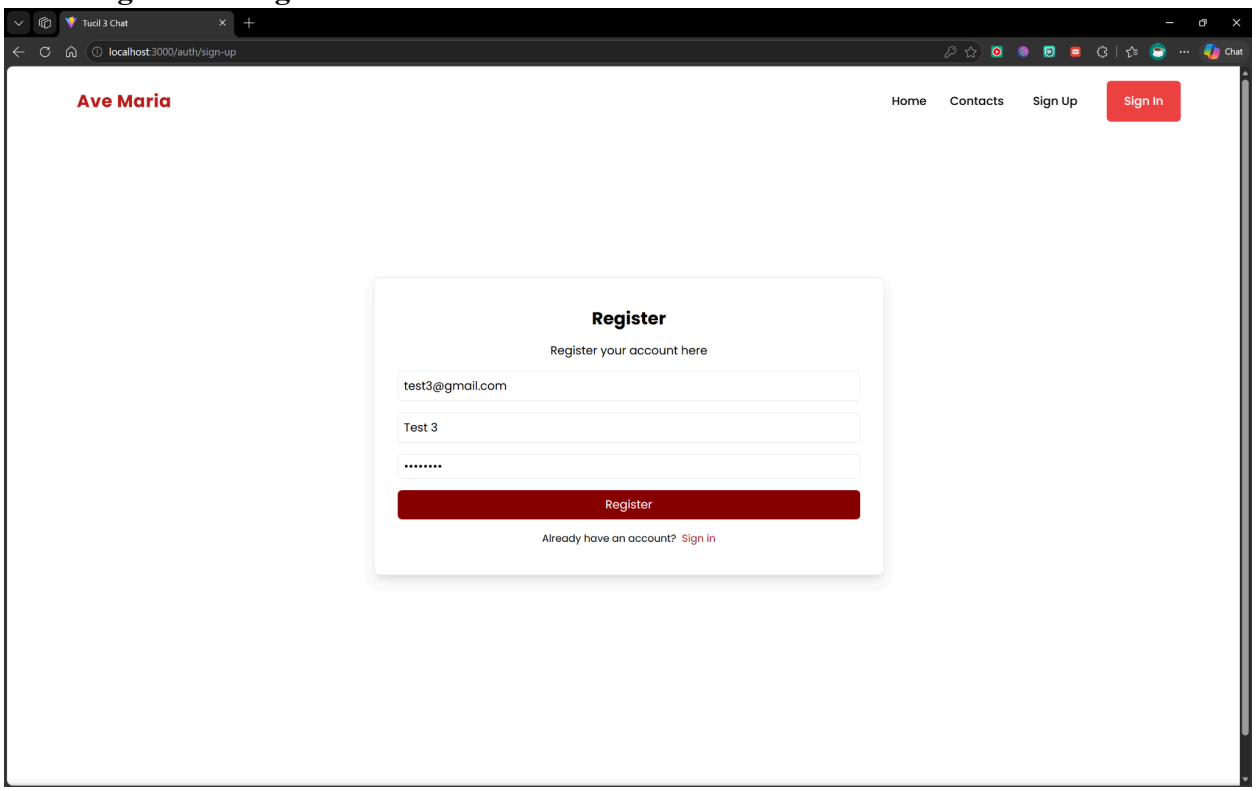
Gambar 2.12 After Sending Message

BAB III

PENGUJIAN PROGRAM DAN ANALISIS HASIL

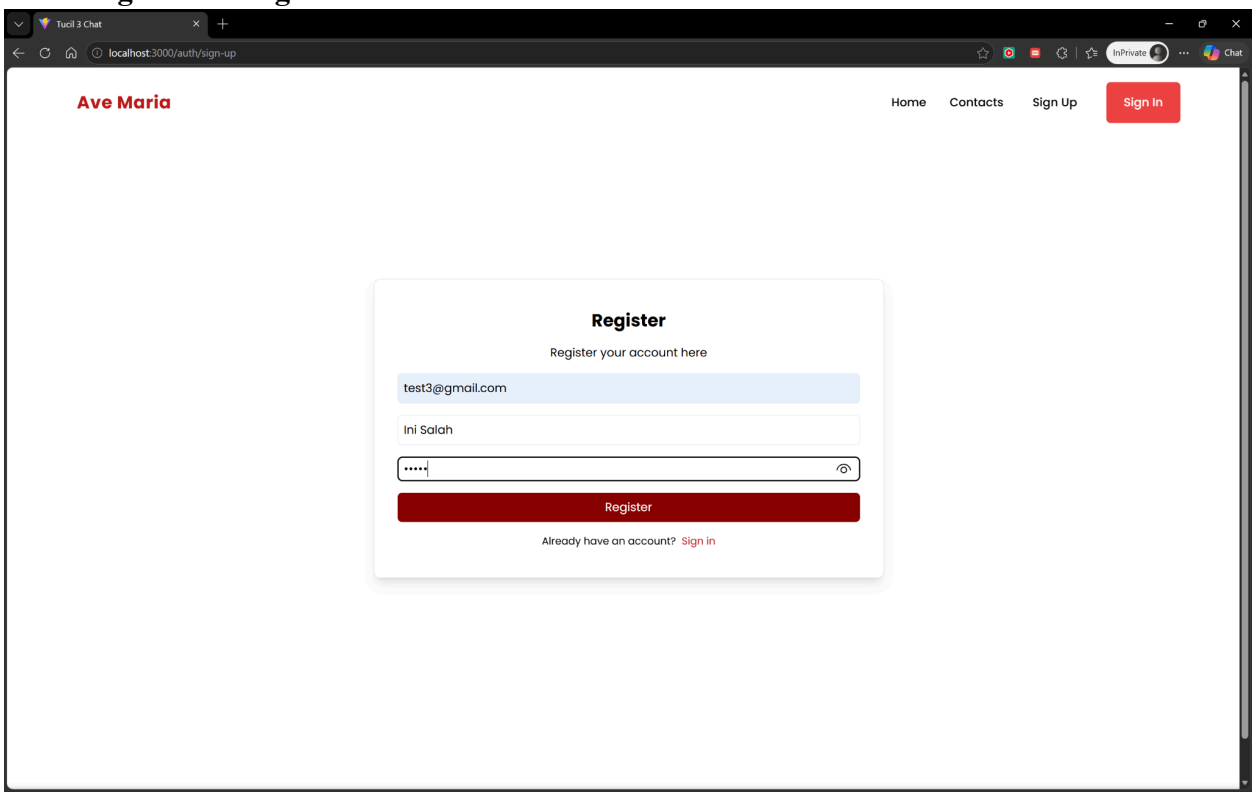
3.1. Uji Autentikasi dan Manajemen Pengguna

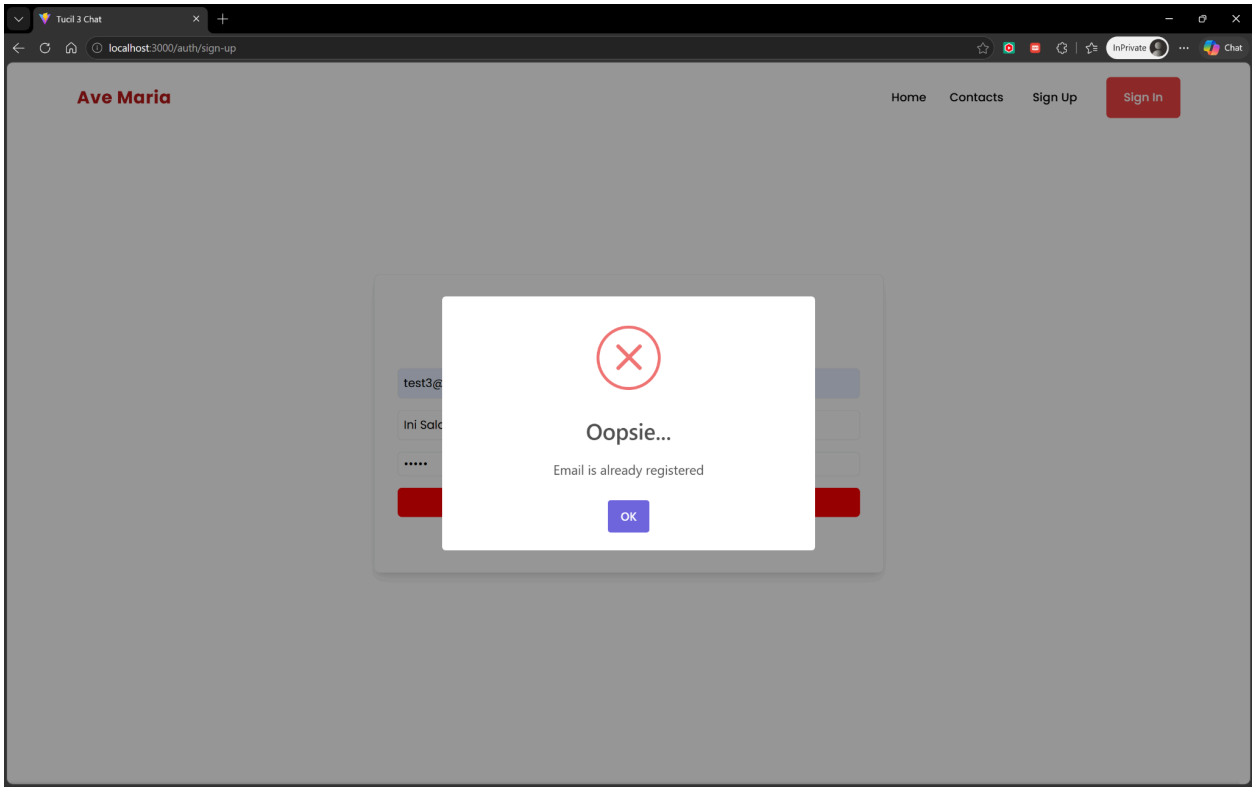
3.1.1. Registrasi dengan Data Valid



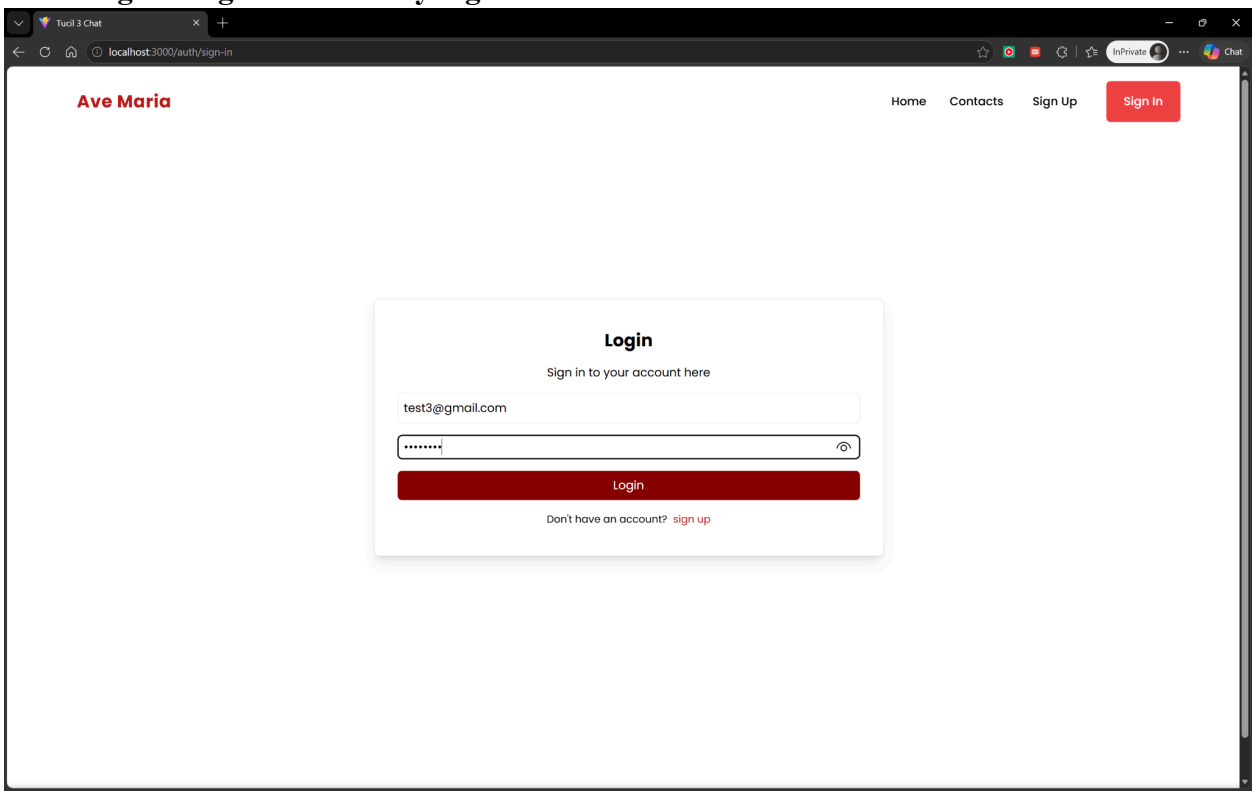

```
Enterprise Tucil3> db.Users.find({ username: "Test 3" }).pretty()
[
  {
    _id: ObjectId('6a033601a15bab198bab958d'),
    email: 'test3@gmail.com',
    password: '$2a$09$fQOhrDaMXj922bSCThr77O5Gds6LZKNkMjdn36jDz1CEpt/ldHckK',
    username: 'Test 3',
    publicKey: 'MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAE6KfCA7KKHH5kyxDdtxrEByeptDBwgVdc
RYal6eQZXEHhxdT4d4lgu4d48wmRO4kGaOUN7IbUtDDZQX+tIZ9lWA==',
    encryptedPrivateKey: '{"iv":"9CvF5G9qmZUqTXq7","ciphertext":"6zn9VDP7LHALFGdQh++
ao4EEOdAgP0e72WJrn+79F+WVmIr8CwXvvsAJ1Dz1GH1abUIal5qk4/sjj7Wv7Exrme6wej9IfFAqLWC1wD/
B85y8hTDYH/WKII8rVQ6bAJJGHHcg4vhnN5R7wkRU6A7wGSG9rk73X2r138SX32lmiFkkuCPh4nWR0AUpkQH
9RzDusaOjwRhoQAv0jQ==}',
    salt: 'Zk8fsSUB4SU2VLvpltaZTQ=='
  }
]
```

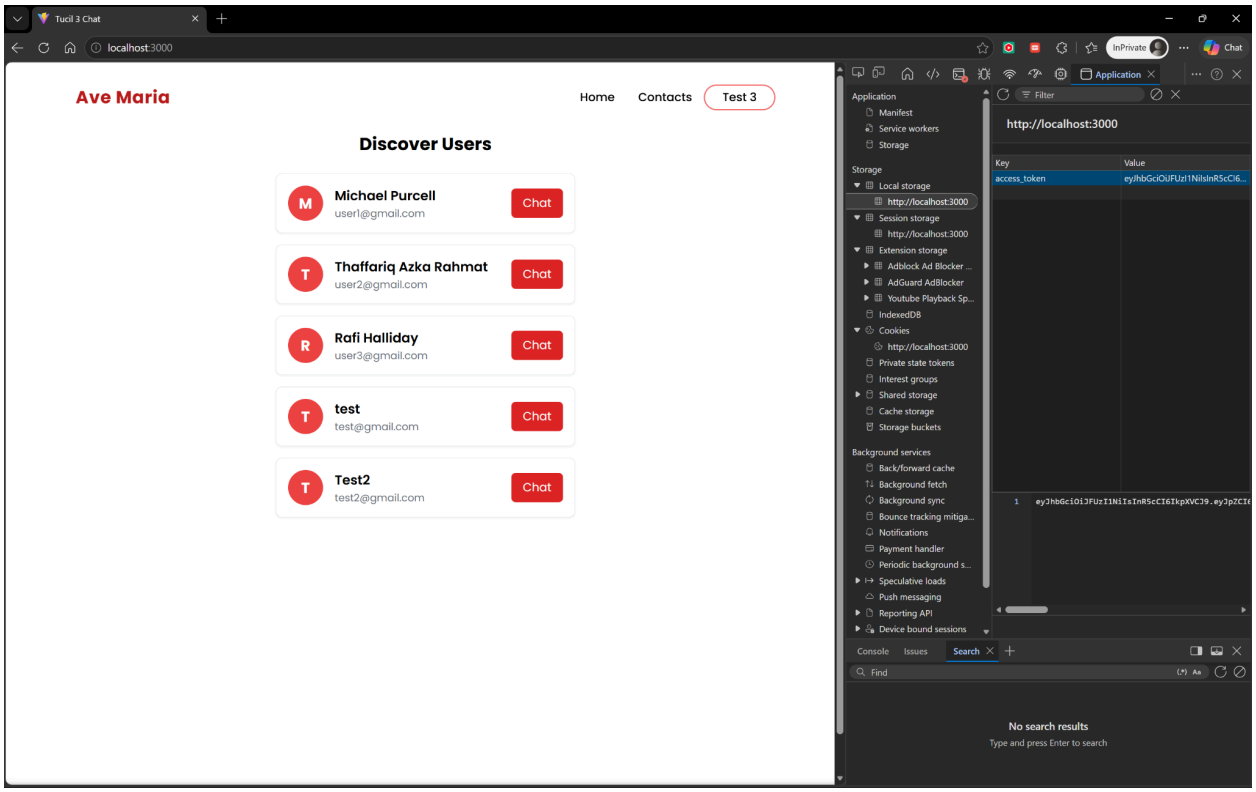
3.1.2. Registrasi dengan Email Terdaftar





3.1.3. Login dengan Password yang Benar

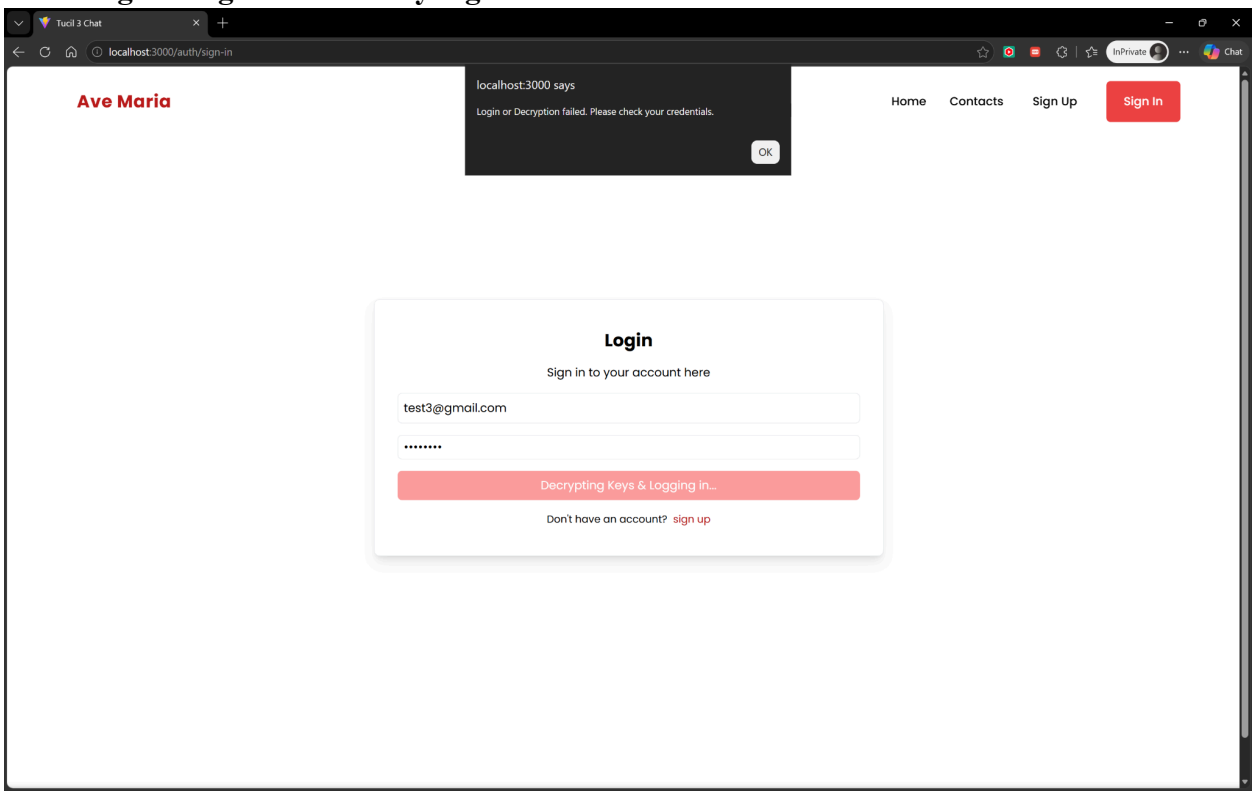




JWT untuk sesi ini:


```
eyJhbGciOiJFUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZhMDMzNjAxYTE1YmFiMT
k4YmFiOTU4ZCIsImVtYWlsIjoidGVzdDNAZ21haWwuY29tIiwidXNlcm5hbWUiOiJUZ
XN0IDMiLCJpYXQiOjE3Nzg1OTU1NzMsImV4cCI6MTc3ODY4MTk3M30.MEUCIDFaScq1
qjAHeC6qXrpr5CA28J34N_7rR1A9rK5SeKfdAiEA3Fy1Y3ckXgaqBTApt99K5oKo9kg
3ZhhDbDY86RwiikM
```

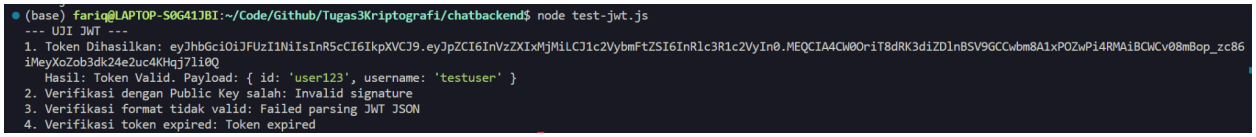

3.1.4. Login dengan Password yang Salah



3.2. Uji Implementasi dan JSON Web Token (JWT)

Pengujian ini memuat uji membaca payload valid, memverifikasi public key yang salah, mengetes pesan error dengan format token invalid, dan mengetes klaim waktu token yang tidak sesuai.

Berikut pengujiannya:



Berikut cuplikan kode script yang digunakan untuk menguji:

```
const { sign, verify } = require("./helpers/jwt");
```

```
const crypto = require("crypto");

const { publicKey, privateKey } = crypto.generateKeyPairSync('ec', {
  namedCurve: 'prime256v1',
  publicKeyEncoding: { type: 'spki', format: 'pem' },
  privateKeyEncoding: { type: 'pkcs8', format: 'pem' },
});

const runJwtTest = () => {
  console.log("--- UJI JWT ---");
  const payload = { id: "user123", username: "testuser" };

  const token = sign(payload, privateKey);
  console.log("1. Token Dihasilkan:", token);
  try {
    const decoded = verify(token, publicKey);
    console.log("   Hasil: Token Valid. Payload:", decoded.payload);
  } catch (err) {
    console.error("   Hasil: Gagal verifikasi token valid!", err.message);
  }

  const { publicKey: wrongPublicKey } = crypto.generateKeyPairSync('ec', {
namedCurve: 'prime256v1', publicKeyEncoding: { type: 'spki', format: 'pem' }
});
  try {
    verify(token, wrongPublicKey);
  } catch (err) {
    console.log("2. Verifikasi dengan Public Key salah:", err.message); //
Expected: Invalid signature
  }

  try {
    verify("format.salah.aja", publicKey);
  } catch (err) {
    console.log("3. Verifikasi format tidak valid:", err.message); //
Expected: Failed parsing atau Invalid format
  }

  const expiredClaims = { ...payload, exp: Math.floor(Date.now() / 1000) - 10
};
  const expiredToken = sign(expiredClaims, privateKey);
  try {
    verify(expiredToken, publicKey);
  } catch (err) {
    console.log("4. Verifikasi token expired:", err.message); // Expected:
Token expired
  }
};

runJwtTest();
```

3.3. Uji Pembentukan Kunci Komunikasi (ECDH dan KDF)

Pengujian ini memuat uji key exchange yang menghasilkan shared secret yang sama dan diturunkan menjadi kunci simetris yang digunakan untuk enkripsi pesan.
Berikut pengujiannya:

```
(base) fariq@LAPTOP-S0G41JBI:~/Code/Github/Tugas3Kriptografi/chatbackend$ node test-crypto.js
--- UJI ECDH & KDF ---
1. Shared Secret Alice (Hex): 558ff472dfc3a585cd1d019a1f2515951c5cd692a047cc08c72ae79135fbda40
2. Shared Secret Bob   (Hex): 558ff472dfc3a585cd1d019a1f2515951c5cd692a047cc08c72ae79135fbda40
   Hasil: Secrets Match? true
3. Kunci Simetris Alice (B64): 3DKt+hhXXDpjX4yvRFCUzCetpRP4phLdJpArV9XVBWc=
4. Kunci Simetris Bob   (B64): 3DKt+hhXXDpjX4yvRFCUzCetpRP4phLdJpArV9XVBWc=
   Hasil: Kunci AES Cocok? true
```

Berikut cuplikan kode script yang digunakan untuk menguji:

```
const crypto = require("crypto");

console.log("--- UJI ECDH & KDF ---");

const alice = crypto.createECDH('prime256v1');
alice.generateKeys();
const bob = crypto.createECDH('prime256v1');
bob.generateKeys();

const aliceSecret = alice.computeSecret(bob.getPublicKey());
const bobSecret = bob.computeSecret(alice.getPublicKey());

console.log("1. Shared Secret Alice (Hex):", aliceSecret.toString('hex'));
console.log("2. Shared Secret Bob   (Hex):", bobSecret.toString('hex'));
console.log("   Hasil: Secrets Match?", aliceSecret.equals(bobSecret));

const deriveKey = (secret) => {
  const derived = crypto.hkdfSync('sha256', secret, Buffer.alloc(0),
Buffer.alloc(0), 32);
  return Buffer.from(derived);
};

const aliceAesKey = deriveKey(aliceSecret);
const bobAesKey = deriveKey(bobSecret);

console.log("3. Kunci Simetris Alice (B64):", aliceAesKey.toString('base64'));
console.log("4. Kunci Simetris Bob   (B64):", bobAesKey.toString('base64'));
console.log("   Hasil: Kunci AES Cocok?", aliceAesKey.equals(bobAesKey));
```

3.4. Uji Enkripsi dan Dekripsi Pesan

Pengujian ini memuat uji plainteks hasil dekripsi pesan yang berhasil dan pesan error dekripsi gagal.
Berikut pengujiannya:

```
● (base) fariq@LAPTOP-S0G41JBI:~/Code/Github/Tugas3Kriptografi/chatbackend$ node test-decryption.js

  --- UJI ENKRIPSI & DEKRIPSI ---
  1. Pesan Asli: Halo, ini pesan rahasia!
  2. Ciphertext: aAW/3voj1YHkwQBKXnPqqz+L1ZafEFo8
  3. Hasil Dekripsi (Kunci Benar): Halo, ini pesan rahasia!
  4. Dekripsi dengan Kunci Salah: [Decryption Failed]
```

Berikut cuplikan kode script yang digunakan untuk menguji:

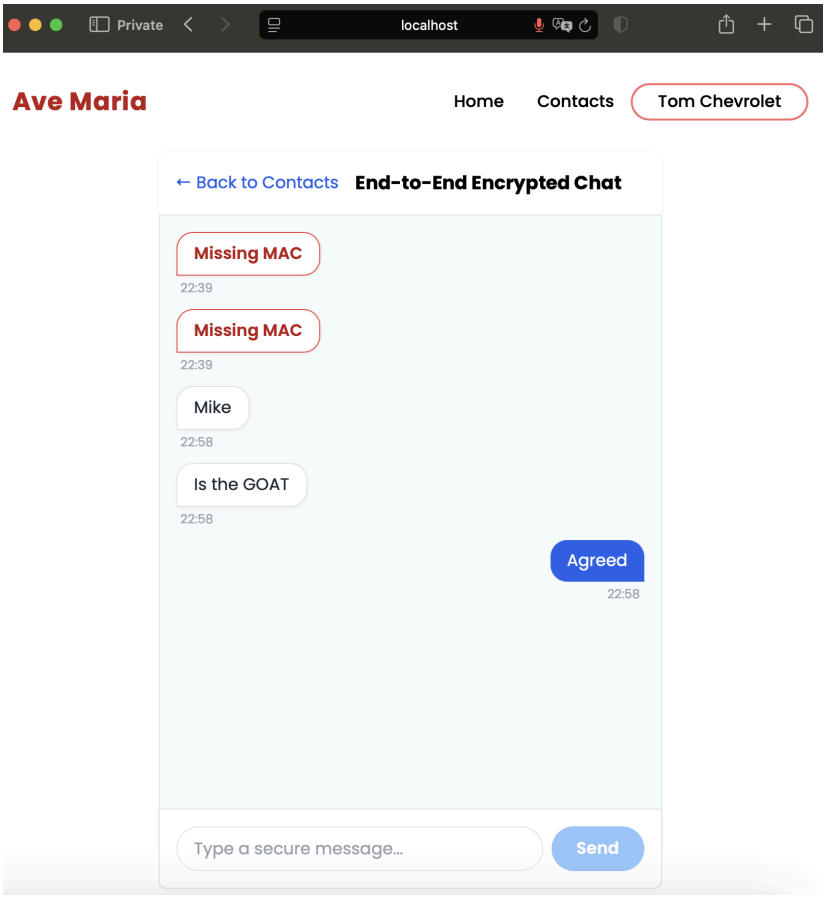
```
const crypto = require("crypto");

const runEncryptionTest = () => {
  console.log("\n--- UJI ENKRIPSI & DEKRIPSI ---");
  const sharedKey = crypto.randomBytes(32); // Kunci hasil KDF
  const plainText = "Halo, ini pesan rahasia!";

  const iv = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv('aes-256-gcm', sharedKey, iv);
  let ciphertext = cipher.update(plainText, 'utf8', 'base64');
  ciphertext += cipher.final('base64');
  const authTag = cipher.getAuthTag();

  console.log("1. Pesan Asli:", plainText);
  console.log("2. Ciphertext:", ciphertext);

  try {
    const decipher = crypto.createDecipheriv('aes-256-gcm', sharedKey, iv);
    decipher.setAuthTag(authTag);
    let decrypted = decipher.update(ciphertext, 'base64', 'utf8');
    decrypted += decipher.final('utf8');
    console.log("3. Hasil Dekripsi (Kunci Benar):", decrypted);
  } catch (err) {
    console.error("3. Dekripsi Gagal padahal kunci benar!");
  }

  const wrongKey = crypto.randomBytes(32);
  try {
    const decipherWrong = crypto.createDecipheriv('aes-256-gcm', wrongKey,
iv);
    decipherWrong.setAuthTag(authTag);
    decipherWrong.update(ciphertext, 'base64', 'utf8');
    decipherWrong.final('utf8');
  } catch (err) {
    console.log("4. Dekripsi dengan Kunci Salah:", "[Decryption Failed]");
  }
};

runEncryptionTest();
```

Bagian enkripsi menggunakan Web Crypto API, sehingga script ini hanya menguji logika yang dipakai pada enkripsi website.

3.5. Uji Integritas dan Autentikasi Pesan (MAC) (Bonus)

a. Kirim pesan dengan MAC yang valid, perlihatkan bahwa pesan diterima dan diproses dengan benar.
b. Ubah sebagian isi pesan atau MAC, perlihatkan bahwa verifikasi gagal dan pesan ditandai tidak valid sebelum didekripsi.

Untuk kedua kasus ini, bisa ditunjukkan di dalam screenshot ini:



Message yang memiliki tulisan "Missing MAC" adalah chat yang tersimpan di dalam database sebelum implementasi MAC sepenuhnya terealisasi. Sebelum fungsi MAC selesai, dua pesan ini masih bisa dilihat isinya. Entry kedua message di dalam DB Mongo tidak memiliki MAC. Setelah implementasi, kedua pesan tersebut tidak bisa lagi dibaca karena tidak memiliki MAC (yang sekarang diperlukan). Sementara, tiga pesan berikutnya, yang dikirimkan setelah implementasi MAC selesai, bisa dilihat tanpa adanya masalah.

BAB IV

KESIMPULAN

Sejauh percobaan dan pengujian yang sudah dilakukan, klien percakapan Ave Maria ini sudah aman. Fitur keamanan yang "biasa" seperti autentikasi pengguna sudah berjalan dengan baik. Pengguna tidak bisa mengakses percakapan tanpa masuk menggunakan akun yang benar. Layanan kriptografi yang digunakan membuat aplikasi ini menjadi benar-benar aman (sekali lagi, sejauh pengamatan kami).

Autentikasi dengan JWT memastikan beberapa hal:

- Kevalidan payload
- Kevalidan kunci publik
- Kevalidan format dan klaim token

Sehingga percakapan tidak sekadar terjadi karena "kelihatannya" valid saja melainkan memiliki metode kriptografis yang menjaminnya (untuk kasus token ini dengan ECDSA)

Pembentukan kunci dengan ECDH dan KDF memberikan aplikasi keamanan dan performa yang baik. Handshake bisa dilakukan relatif cepat (dibandingkan dengan memakai algoritma asimetris lain seperti RSA) karena memakai kurva eliptik. Kunci yang diturunkan berhasil mengamankan plaintext yang dikirimkan oleh pengguna. Ketika kunci yang dipakai salah, pesan tidak akan bisa didekripsi.

Terakhir MAC memastikan bahwa segala serangan pengubahan akan terdeteksi. Kode akan dibandingkan dengan hasil hash di sisi penerima dan segala perubahan di tengah transmisi akan mengakibatkan ketidakcocokan. Pemakaian SHA256 menjamin bahwa kemungkinan kolisi terlalu kecil sehingga bisa dibilang nihil.

Aplikasi ini tahan terhadap semua serangan berikut:

1. Kriptanalisis secara brute force: AES256 tidak bisa dikriptanalisis secara brute force dalam waktu yang masuk akal
2. Kriptanalisis dengan metode statistika: confusion dan diffusion dalam AES256 sudah cukup baik sehingga pola statistik akan terhapus
3. Man-in-the-middle: Pemakaian ECDSA pada JWT melindungi aplikasi dari serangan di tengah karena ECDSA memastikan bahwa keabsahan suatu token bisa dicek, MAC memiliki fungsi yang serupa
4. Keterjagaan kunci simetris: Pemakaian ECDH untuk menurunkan kunci simetris (yang akan dipakai AES) memastikan kunci simetris bisa tetap didapat walau melalui saluran yang tidak aman

DAFTAR PUSTAKA

Web Crypto API - Web APIs | MDN. (2024, September 2). https://developer.mozilla.org/en-US/docs/Web/API/Web_Crypto_API

RFC 7519: JSON Web Token (JWT). (n.d.). IETF Datatracker. https://datatracker.ietf.org/doc/html/rfc7519

jsonwebtoken. (n.d.). Npm. https://www.npmjs.com/package/jsonwebtoken

LAMPIRAN

Lampiran 1. Pernyataan Integritas

Kami menyatakan bahwa kode program yang dihasilkan bukan merupakan hasil salinan mentah (raw output) dari Generative AI, melainkan hasil pengembangan dan penulisan mandiri.

Thaffariq Azka Rahmat	Michael Ballard Isaiah Silaen	Muhammad Rafi Ramadan

Lampiran 2. Pranala Repositori Github
https://github.com/Michael-Ballard-Isaiah-Silaen/Tugas3Kriptografi.git

Lampiran 3. Pranala Video Demo
https://youtu.be/luL8GeMFVmI

Lampiran 4. Pembagian Tugas

Nama	NIM	Tugas
Thaffariq Azka Rahmat	18223048	• Membuat kode bagian JWT • Membuat laporan bagian Bab 2 Implementasi Program, Bab 3 Pengujian Program dan Analisis Hasil, dan melakukan formatting • Membuat script pengujian program
Michael Ballard Isaiah Silaen	18223080	• Membuat chat app dasar • Membuat feature tambahan secure messaging • Melakukan konfigurasi docker dan konfigurasi lain • Membuat laporan bagian Bab 3 Pengujian Program dan Analisis Hasil bagian MAC
Muhammad Rafi Ramadhan	18223094	• Membuat kode bagian ECDH, KDF, AES • Membuat laporan bagian Bab 1 Landasan Teori, Bab 2 Perancangan Program, dan Bab 4 Kesimpulan • Mendesain UI program